

به نام خدا



Smart Car گزارش کار پروژه

درس

سیستم های نهفته و بی درنگ

TinyOD-STM32

سبحان نصیری

استاد

دکتر سیاوش زرگری

سال تحصیلی

۱۴۰۴-۲

گزارش کار پروژه فاز اول: پیاده‌سازی و آموزش مدل یادگیری ماشین برای تشخیص مانع

این فاز شامل مراحل کامل ساخت، آموزش و استخراج یک مدل یادگیری ماشین بسیار سبک برای استفاده در یک MCU (Microcontroller Unit) واحد میکروکنترلر (است). کدها به صورت ماژولار نوشته شده‌اند و مراحل به صورت زیر پیش می‌روند.

__pycache__	۱۴۰۵/۰۴/۱۲ ب.ظ ۰۷:۵۰	File folder	
dataset	۱۴۰۵/۰۳/۱۶ ق.ظ ۰۱:۱۹	File folder	
best_tiny_model.pth	۱۴۰۵/۰۴/۲۵ ق.ظ ۱۲:۱۸	PTH File	11 KB
data_loader.py	۱۴۰۵/۰۳/۱۶ ق.ظ ۱۲:۵۹	Python Source File	2 KB
export_onnx.py	۱۴۰۵/۰۳/۲۶ ب.ظ ۰۹:۱۰	Python Source File	1 KB
generate_data.py	۱۴۰۵/۰۳/۱۶ ق.ظ ۰۱:۱۶	Python Source File	2 KB
model_F4.py	۱۴۰۵/۰۴/۰۲ ب.ظ ۰۸:۳۴	Python Source File	3 KB
model_H7.py	۱۴۰۵/۰۳/۱۶ ق.ظ ۰۱:۲۶	Python Source File	2 KB
obstacle_model.onnx	۱۴۰۵/۰۳/۲۶ ب.ظ ۰۹:۵۱	ONNX File	6 KB
QN.py	۱۴۰۵/۰۳/۲۶ ب.ظ ۰۹:۳۷	Python Source File	2 KB
test.py	۱۴۰۵/۰۴/۱۲ ب.ظ ۰۷:۵۲	Python Source File	3 KB
train.py	۱۴۰۵/۰۳/۲۶ ب.ظ ۰۹:۵۱	Python Source File	4 KB
train_F4.py	۱۴۰۵/۰۳/۲۶ ب.ظ ۰۹:۰۳	Python Source File	4 KB

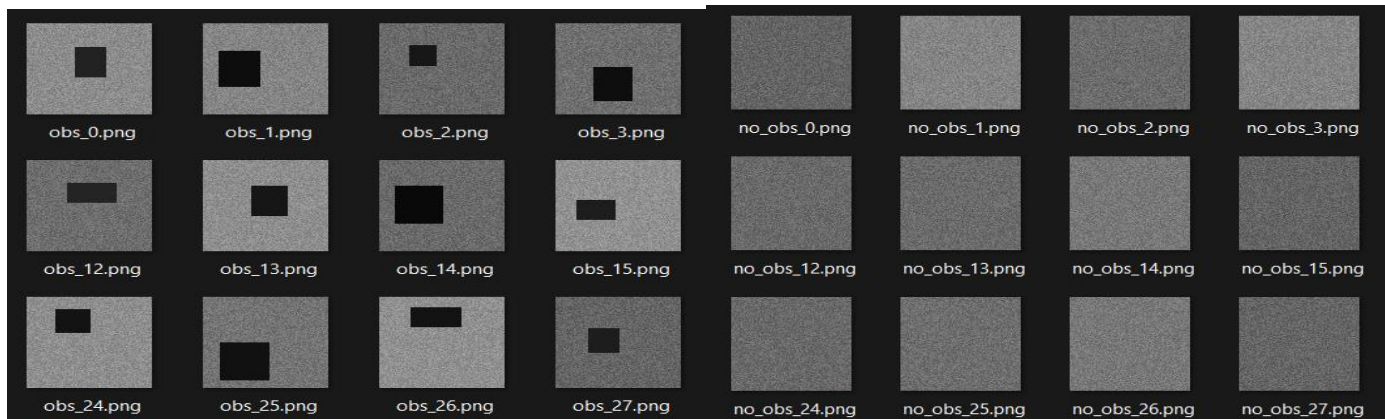
مرحله ۱: تولید داده‌های آموزشی (Synthetic Data Generation)

در فایل `generate_data.py`، از آنجایی که مجموعه داده واقعی وجود ندارد، برنامه‌ای برای تولید ۲۰۰ تصویر برای هر کلاس به صورت تصادفی نوشته شده است. کلاس صفر

(`class_0_no_obstacle`) فقط شامل پس‌زمینه همراه با نویز است و کلاس یک

(`class_1_obstacle`) علاوه بر پس‌زمینه، شامل یک مربع تیره رنگ است که نقش مانع را ایفا

می‌کند.



مرحله ۲: آماده‌سازی و بارگذاری داده‌ها (Data Loading and Preprocessing)

در فایل `data_loader.py`، کلاس `TinyObstacleDataset` تصاویر تولید شده را می‌خواند. تمامی تصاویر توسط کتابخانه تصویر پایتون خوانده شده و سپس توسط تبدیل گرهای به ابعاد ۹۶ در ۹۶ تغییر اندازه داده می‌شوند و به مقیاس خاکستری (Grayscale) تبدیل می‌گردند تا میزان RAM - Random Access Memory (حافظه دسترسی تصادفی) کمتری هنگام پردازش اشغال شود و سرعت مدل بالا برود.

مرحله ۳: معماری شبکه عصبی (Model Architecture)

در فایل `model_F4.py`، یک - Convolutional Neural Network (CNN) شبکه عصبی پیچشی (اختصاصی به نام `TinyObstacleNetMCU` طراحی شده است. برای بهینه‌سازی مدل جهت اجرا روی میکروکنترلرهایی با محدودیت پردازشی شدید، از لایه‌های Depthwise (DSC) - Separable Convolution پیچش تفکیک‌پذیر عمقی (استفاده شده است. این نوع لایه، محاسبات را به جای یک مرحله‌ی سنگین، در دو مرحله‌ی سبک انجام می‌دهد که حجم مدل را به شدت کاهش می‌دهد. در نهایت از تابع سیگموئید برای خروجی احتمال بین ۰ تا ۱ استفاده شده است.

مرحله ۴: آموزش مدل (Model Training)

در فایل `train.py`، داده‌ها به صورت تصادفی به نسبت ۸۰ به ۲۰ به دو بخش آموزش (Train) و اعتبارسنجی (Validation) تقسیم می‌شوند. شبکه با استفاده از تابع هزینه BCE (Binary Cross Entropy - آنتروپی متقاطع دوتایی) (و بهینه‌ساز Adam (Adaptive Moment Estimation - تخمین گشتاور تطبیقی) (در ۱۰ دوره (Epoch) متوالی آموزش می‌بیند. در هر مرحله اگر خطای مدل روی داده‌های اعتبارسنجی کمتر از قبل باشد، مدل ذخیره می‌شود.

```
Sobhan torch_env Ai_Model main 3.10.19 8.833s python .\train.py
[*] Training on device: cuda
[*] Starting Training Loop...
Epoch [1/10] Train Loss: 0.6755, Train Acc: 60.62% | Val Loss: 0.7066, Val Acc: 40.00%
-> Best model saved with Val Loss: 0.7066
Epoch [2/10] Train Loss: 0.6498, Train Acc: 70.62% | Val Loss: 0.6935, Val Acc: 40.00%
-> Best model saved with Val Loss: 0.6935
Epoch [3/10] Train Loss: 0.6245, Train Acc: 75.62% | Val Loss: 0.6776, Val Acc: 66.25%
-> Best model saved with Val Loss: 0.6776
Epoch [4/10] Train Loss: 0.5945, Train Acc: 81.25% | Val Loss: 0.6550, Val Acc: 71.25%
-> Best model saved with Val Loss: 0.6550
Epoch [5/10] Train Loss: 0.5655, Train Acc: 85.94% | Val Loss: 0.6145, Val Acc: 96.25%
-> Best model saved with Val Loss: 0.6145
Epoch [6/10] Train Loss: 0.5329, Train Acc: 92.50% | Val Loss: 0.5655, Val Acc: 97.50%
-> Best model saved with Val Loss: 0.5655
Epoch [7/10] Train Loss: 0.5070, Train Acc: 93.75% | Val Loss: 0.5277, Val Acc: 97.50%
-> Best model saved with Val Loss: 0.5277
Epoch [8/10] Train Loss: 0.4730, Train Acc: 99.38% | Val Loss: 0.5027, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.5027
Epoch [9/10] Train Loss: 0.4467, Train Acc: 99.38% | Val Loss: 0.4746, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.4746
Epoch [10/10] Train Loss: 0.4207, Train Acc: 100.00% | Val Loss: 0.4497, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.4497
[*] Training complete. Best model saved to best_tiny_model.pth
```

مرحله ۵: استخراج و آماده‌سازی برای سخت‌افزار (Model Exporting)

در فایل `export_onnx.py`، بهترین مدل آموزش‌دیده (فایل `pth`) بارگذاری شده و به فرمت استاندارد - ONNX (Open Neural Network Exchange) تبادل شبکه عصبی (باز) تبدیل می‌شود. این فرمت استاندارد به ما اجازه می‌دهد مدل را از پایتورچ خارج کرده و توسط ابزارهای سخت‌افزاری به زبان C برای میکروکنترلر ترجمه کنیم.

گزارش کار پروژه: فاز دوم - پیاده‌سازی سخت‌افزاری مدل هوش مصنوعی و رابط کاربری

در فاز دوم این پروژه، مدل آموزش‌دیده‌ی مرحله‌ی قبل را به یک سخت‌افزار واقعی (میکروکنترلر) منتقل می‌کنیم. این فاز شامل پیکربندی سخت‌افزار، استقرار مدل با استفاده از ابزارهای توسعه‌ی سازنده، راه‌اندازی نمایشگر برای نشان دادن خروجی بصری و تنظیم پورت سریال برای دیباگ - Debug (اشکال‌زدایی) است.

مرحله ۱: انتخاب سخت‌افزار و تنظیمات کلاک (MCU & Clock Configuration)

برای این پروژه از میکروکنترلر STM32F401VETx با پکیج LQFP100 استفاده شده است. این تراشه بر پایه هسته ARM Cortex-M4 است و قدرت پردازشی مناسبی برای اجرای مدل‌های سبک یادگیری ماشین دارد. کلاک سیستم با استفاده از منبع - HSE (High Speed External) نوسان‌ساز خارجی سرعت بالا (و واحد - PLL) Phase-Locked Loop (حلقه قفل فاز) بر روی حداکثر فرکانس ممکن یعنی 16 مگاهرتز (16 MHz) تنظیم شده است تا محاسبات ماتریسی شبکه عصبی با بیشترین سرعت انجام شود.

از آنجایی که در محیط شبیه‌ساز پروتئوس اتصال مستقیم دوربین به میکروکنترلر بسیار پیچیده است، ما ۵ تصویر مختلف را به عنوان ورودی‌های تست (Inference Test) انتخاب کردیم. با استفاده از یک اسکرپت پایتون، تصاویر ۹۶ در ۹۶ پیکسل سیاه و سفید خوانده شده و به آرایه‌های یک‌بعدی هگزادسیمال (Hexadecimal) در قالب زبان C تبدیل شدند. این آرایه‌ها در حافظه فلش میکروکنترلر ذخیره می‌شوند تا مستقیماً به ورودی شبکه عصبی تزریق شوند.

 CreateIMG.py	۱۴۰۵/۰۳/۲۶ ب.ظ ۱۰:۲۲	Python Source File	1 KB
 image_data.txt	۱۴۰۵/۰۴/۱۳ ب.ظ ۱۰:۱۸	Text Document	56 KB
 obs_0.png	۱۴۰۵/۰۳/۱۶ ق.ظ ۰۱:۱۹	PNG File	6 KB

مرحله ۴: راه‌اندازی نمایشگر گرافیکی ILI9341 با استفاده از SPI

برای نمایش تصاویر و نتایج، از یک - Liquid Crystal Display (LCD نمایشگر کریستال مایع) با درایور ILI9341 استفاده شد. ارتباط با این نمایشگر از طریق پروتکل Serial (SPI Peripheral Interface - رابط محیطی سریال) برقرار می‌شود. تنظیمات SPI1 به صورت ۸ بیتی، - Most Significant Bit (MSB First) با ارزش‌ترین بیت در ابتدا، با Prescaler برابر ۲ تنظیم شد CPOL Low, CPHA 1 Edge پایه‌های PA5 برای SCK، پایه PA7 برای MOSI اختصاص یافتند.

همچنین پایه‌های PA4، PA6 و PA8 برای کنترل تراشه (CS)، تعیین دیتا/کامند (DC) و ریست (RST) در نظر گرفته شدند.

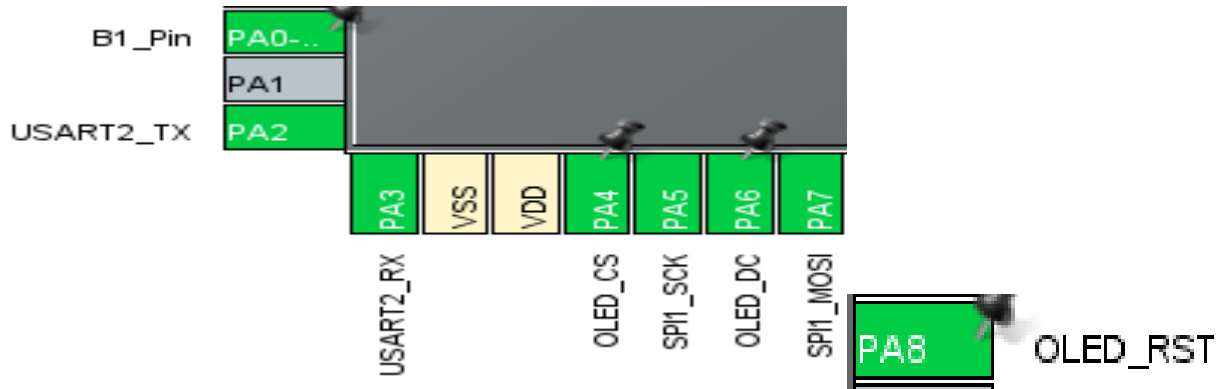
مرحله ۵: راه‌اندازی UART برای دیباگ در ترمینال مجازی (Virtual Terminal)

جهت چاپ لاگ‌های سیستم، درصد اطمینان خروجی شبکه عصبی (Confidence) و زمان اجرای مدل (Inference Time)، از پروتکل - UART (Universal Asynchronous Receiver-Transmitter فرستنده-گیرنده ناهمگام جهانی) استفاده شد. واحد USART2 با سرعت انتقال داده (Baud Rate) برابر با ۹۶۰۰ بیت بر ثانیه، ۸ بیت داده و بدون بیت توازن (Parity None)

تنظیم گردید. پایه‌های PA2 (TX) و PA3 (RX) به ترمینال مجازی پروتئوس متصل می‌شوند.

همچنین وقفه (Interrupt) مربوط به USART2 در بخش NVIC (Nested Vectored

Interrupt Controller - کنترل‌کننده وقفه برداری تودرتو (فعال شد تا دریافت داده‌ها بدون متوقف کردن برنامه اصلی انجام شود).



Basic Parameters

Frame Format	Motorola
Data Size	8 Bits
First Bit	MSB First

Clock Parameters

Prescaler (for Baud Rate)	2
Clock Polarity (CPOL)	Low
Clock Phase (CPHA)	1 Edge

Basic Parameters

Baud Rate	9600 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

Time base: System tick timer	☒
PVD interrupt through EXTI line 16	☐
Flash global interrupt	☐
RCC global interrupt	☐
EXTI line0 interrupt	☒
SPI1 global interrupt	☐
USART2 global interrupt	☒
FPU global interrupt	☐

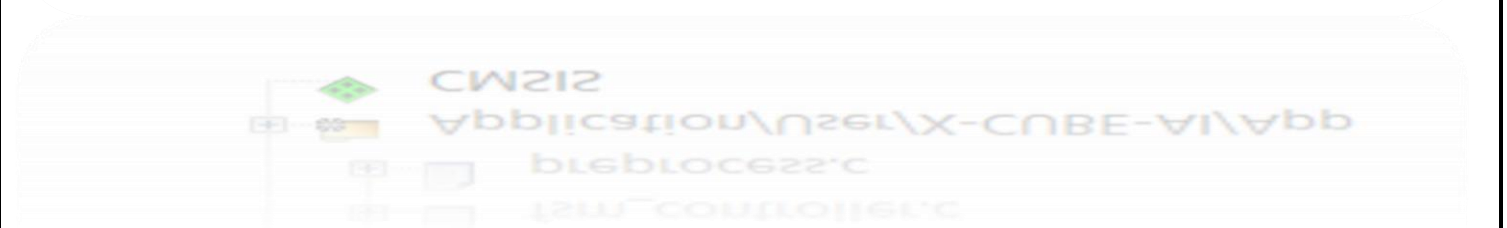
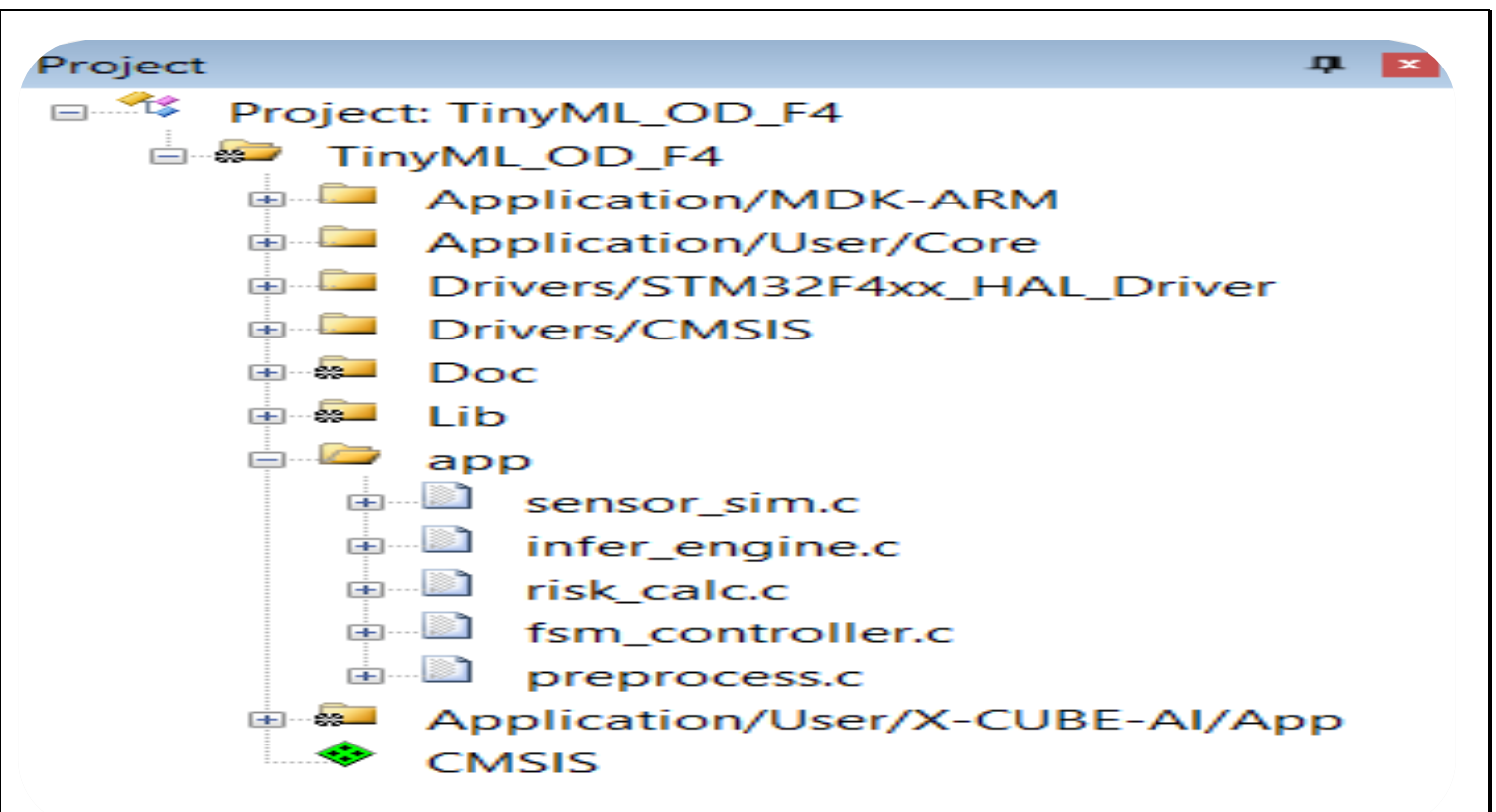
گزارش کار پروژه: فاز سوم - توسعه نرم افزار، ادغام سیستم و منطق تصمیم گیری

در فاز سوم، کدهای نوشته شده ساختاریافته شده و ماژول های مختلف اعم از شبیه ساز سنسور، موتور استنتاج هوش مصنوعی (Inference Engine)، محاسبه گر ریسک (Risk Calculator) و ماشین حالت متناهی (FSM - Finite State Machine) با یکدیگر ادغام - Integration) یکپارچه سازی (می شوند. این فاز هسته اصلی منطق برنامه را شکل می دهد.

Name	Date modified	Type	Size	Name	Date modified	Type	Size
crch	۱۴۰۵/۰۴/۱۴ پ.ظ ۱۲:۵۴	C Header Source F...	2 KB	crc.c	۱۴۰۵/۰۴/۱۴ پ.ظ ۱۲:۵۴	C Source File	2 KB
fsm_controller.h	۱۴۰۵/۰۳/۳۰ پ.ظ ۰۵:۴۷	C Header Source F...	1 KB	gpio.c	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Source File	3 KB
gpio.h	۱۴۰۵/۰۳/۲۶ پ.ظ ۰۹:۵۴	C Header Source F...	2 KB	ili9341.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۹:۳۲	C Source File	7 KB
ili9341.h	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۹:۱۰	C Header Source F...	1 KB	main.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۹:۰۱	C Source File	10 KB
infer_engine.h	۱۴۰۵/۰۴/۱۴ ق.ظ ۱۰:۵۲	C Header Source F...	1 KB	spi.c	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Source File	4 KB
main.h	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Header Source F...	3 KB	stm32f4xx_hal_msp.c	۱۴۰۵/۰۳/۲۴ پ.ظ ۰۳:۲۶	C Source File	3 KB
preprocess.h	۱۴۰۵/۰۳/۲۶ پ.ظ ۱۱:۴۷	C Header Source F...	1 KB	stm32f4xx_it.c	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Source File	6 KB
risk_calch	۱۴۰۵/۰۳/۲۶ پ.ظ ۱۱:۴۷	C Header Source F...	1 KB	system_stm32f4xx.c	۱۴۰۵/۰۲/۲۶ پ.ظ ۰۶:۳۰	C Source File	27 KB
sensor_sim.h	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۸:۰۵	C Header Source F...	1 KB	usart.c	۱۴۰۵/۰۴/۱۰ پ.ظ ۰۸:۵۳	C Source File	4 KB
spi.h	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Header Source F...	2 KB				
stm32f4xx_hal_conf.h	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Header Source F...	20 KB				
stm32f4xx_it.h	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۱	C Header Source F...	2 KB				
usart.h	۱۴۰۵/۰۳/۲۶ پ.ظ ۰۹:۵۴	C Header Source F...	2 KB				
DebugConfig	۱۴۰۵/۰۳/۲۶ پ.ظ ۱۰:۰۵	File folder		.ai	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	File folder	
RTE	۱۴۰۵/۰۳/۲۶ پ.ظ ۱۰:۰۵	File folder		Core	۱۴۰۵/۰۴/۲۳ پ.ظ ۱۰:۱۹	File folder	
TinyML_OD_F4	۱۴۰۵/۰۴/۲۵ ق.ظ ۰۱:۳۰	File folder		Drivers	۱۴۰۵/۰۳/۲۴ پ.ظ ۰۳:۲۶	File folder	
fsm_controller.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۸:۵۷	C Source File	2 KB	MDK-ARM	۱۴۰۵/۰۴/۲۵ ق.ظ ۰۱:۳۰	File folder	
infer_engine.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۹:۲۲	C Source File	5 KB	Middlewares	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	File folder	
preprocess.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۸:۵۲	C Source File	1 KB	X-CUBE-AI	۱۴۰۵/۰۳/۲۶ پ.ظ ۰۹:۵۵	File folder	
risk_calc.c	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۸:۵۶	C Source File	1 KB	.mxproject	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	MXPROJECT File	9 KB
sensor_sim.c	۱۴۰۵/۰۴/۲۳ پ.ظ ۱۰:۳۲	C Source File	2 KB	TinyML_OD_F4.ioc	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	STM32CubeMX	11 KB
startup_stm32f401xe.s	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	S File	25 KB				
TinyML_OD_F4.sct	۱۴۰۵/۰۴/۲۳ پ.ظ ۰۷:۵۶	Windows Script C...	1 KB				
TinyML_OD_F4.uvprojx	۱۴۰۵/۰۴/۲۵ ق.ظ ۰۱:۳۰	AYSON File	95 KB				
TinyML_OD_F4.uvoptx	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۸:۳۲	UVOPTX File	23 KB				
TinyML_OD_F4.uvprojx	۱۴۰۵/۰۴/۲۴ پ.ظ ۰۸:۳۲	µVision5 Project	53 KB				

مرحله ۱: معماری نرم افزار و ماژول بندی (Software Architecture)

به منظور رعایت اصول مهندسی نرم افزار، کدهای پروژه به جای تمرکز و شلوغی در فایل `main.c`، در قالب یک پوشه مجزا سازماندهی شده‌اند. همان‌طور که در تصویر پایین مشخص است، پوشه `app` در ساختار درختی پروژه ایجاد شده و فایل‌های کلیدی مانند `infer_engine.c`، `fsm_controller.c`، `risk_calc.c`، `sensor_sim.c` و `preprocess.c` درون آن قرار گرفته‌اند که باعث افزایش چشمگیر خوانایی و قابلیت نگهداری کد (Maintainability) می‌شود. (از طریق Add group در نرم افزار Keil).



مرحله ۲: پیکربندی حافظه (Memory Configuration)

مدل‌های یادگیری ماشین (Machine Learning) برای ذخیره لایه‌های فعال‌سازی (Activations) و متغیرهای موقت به فضای رم (RAM - Random Access Memory) بالایی نیاز دارند. همچنین توابعی مانند `snprintf` نیازمند فضای پشته (Stack) مناسب هستند. بر اساس تصویر **Screenshot 2026-07-16 012955.png**، مقدار **Stack Size** برابر $0x2000$ (۸ کیلوبایت) و **Heap Size** برابر $0x4000$ (۱۶ کیلوبایت) تنظیم شده است تا از خطای سرریز حافظه (Stack/Heap Overflow) جلوگیری شود.

Option	Value
Stack Configuration	
Stack Size (in Bytes)	0x0000 2000
Heap Configuration	
Heap Size (in Bytes)	0x0000 4000

مرحله ۳: شبیه‌سازی سنسور دوربین (Sensor Simulation)

در فایل `sensor_sim.c`، تابعی برای تولید فریم‌های پویا (Dynamic Frames) نوشته شده است. این تابع با استفاده از دستور `rand()` نویزهای تصادفی پس‌زمینه ایجاد کرده و یک مربع مشکی رنگ را شبیه‌سازی می‌کند که در هر سناریو ابعاد آن بزرگتر می‌شود (نشان‌دهنده نزدیک شدن مانع). پنج سناریو پیاده‌سازی شده است که با فشردن دکمه وضعیت آن‌ها تغییر می‌کند.

مرحله ۴: پیش‌پردازش و موتور استنتاج هوش مصنوعی (Preprocessing & AI Inference)

ابتدا تصاویر خام در تابع `Preprocess_NormalizeFloat` باید به مقادیر اعشاری (Float) تبدیل شوند. سپس در فایل `infer_engine.c`، داده‌ها با استفاده از توابع کتابخانه (X-CUBE-AI) مشخصاً تابع `ai_ob_model_run` به شبکه عصبی تزریق می‌شوند. خروجی این شبکه یک عدد اعشاری بین ۰ و ۱ است که پس از ضرب در ۱۰۰، به عنوان درصد احتمال وجود مانع (Probability) به عنوان خروجی بازگردانده می‌شود.

مرحله ۵: منطق تصمیم‌گیری بر اساس FSM و TTC

منطق اصلی کنترل سیستم بر عهده یک ماشین حالت متناهی (FSM) است که در `fsm_controller.c` قرار دارد. این سیستم دارای ۴ حالت اصلی است: `STATE_NORMAL`، `STATE_WARNING`، `STATE_BRAKE` و `STATE_SAFE_STOP`. در فایل `risk_calc.c`، شاخص زمان تا برخورد (TTC - Time To Collision) با فرمول $(Distance * Speed) / 1000$ محاسبه می‌شود. اگر سیستم در حالت `WARNING` باشد و زمان TTC به زیر ۱۲۰۰ میلی‌ثانیه برسد، وضعیت فوراً به حالت ترمز (`STATE_BRAKE`) تغییر می‌کند. نمایشگر گرافیکی نیز در فایل `main.c` بر اساس همین وضعیت‌ها آپدیت می‌شود.

مرحله ۶: کامپایل و مصرف منابع (Compilation & Resource Usage)

بر اساس تصویر پایین، پروژه در کامپایلر ARM-MDK با موفقیت و بدون هیچ‌گونه خطا یا هشدار (0 Error, 0 Warning) بیلد (Build - ساخته) شده است. حجم کد برنامه (Code Size) تقریباً ۱۹,۶ کیلوبایت و بخش دیتای رم (ZI-data) در حدود ۶۸,۷ کیلوبایت است که برای اجرای شبکه بر روی یک میکروکنترلر سری STM32F4 منطقی است.

```
Rebuild started: Project: TinyML_OD_F4
*** Using Compiler 'V6.24', folder: 'D:\Keil\ARM\ARMCLANG\Bin'
Rebuild target 'TinyML_OD_F4'
assembling startup_stm32f401xe.s...
compiling crc.c...
compiling stm32f4xx_hal_rcc_ex.c...
compiling ili9341.c...
compiling stm32f4xx_hal_flash_ex.c...
compiling stm32f4xx_it.c...
compiling stm32f4xx_hal_msp.c...
compiling stm32f4xx_hal_crc.c...
compiling spi.c...
compiling usart.c...
compiling gpio.c...
compiling stm32f4xx_hal_flash.c...
compiling main.c...
compiling stm32f4xx_hal_rcc.c...
compiling stm32f4xx_hal_flash_ramfunc.c...
compiling stm32f4xx_hal_gpio.c...
compiling stm32f4xx_hal_pwr.c...
compiling stm32f4xx_hal_pwr_ex.c...
compiling system_stm32f4xx.c...
compiling stm32f4xx_hal_dma_ex.c...
compiling stm32f4xx_hal_exti.c...
compiling stm32f4xx_hal_cortex.c...
compiling stm32f4xx_hal.c...
compiling stm32f4xx_hal_dma.c...
compiling sensor_sim.c...
compiling risk_calc.c...
compiling infer_engine.c...
compiling fsm_controller.c...
compiling preprocess.c...
compiling ob_model_data_params.c...
compiling ob_model_data.c...
compiling stm32f4xx_hal_spi.c...
compiling stm32f4xx_hal_uart.c...
compiling ob_model.c...
linking...
Program Size: Code=19608 RO-data=4296 RW-data=3628 ZI-data=68764
FromELF: creating hex file...
"TinyML_OD_F4\TinyML_OD_F4.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

گزارش کار پروژه: فاز چهارم - شبیه‌سازی سخت‌افزاری (proteus 8)

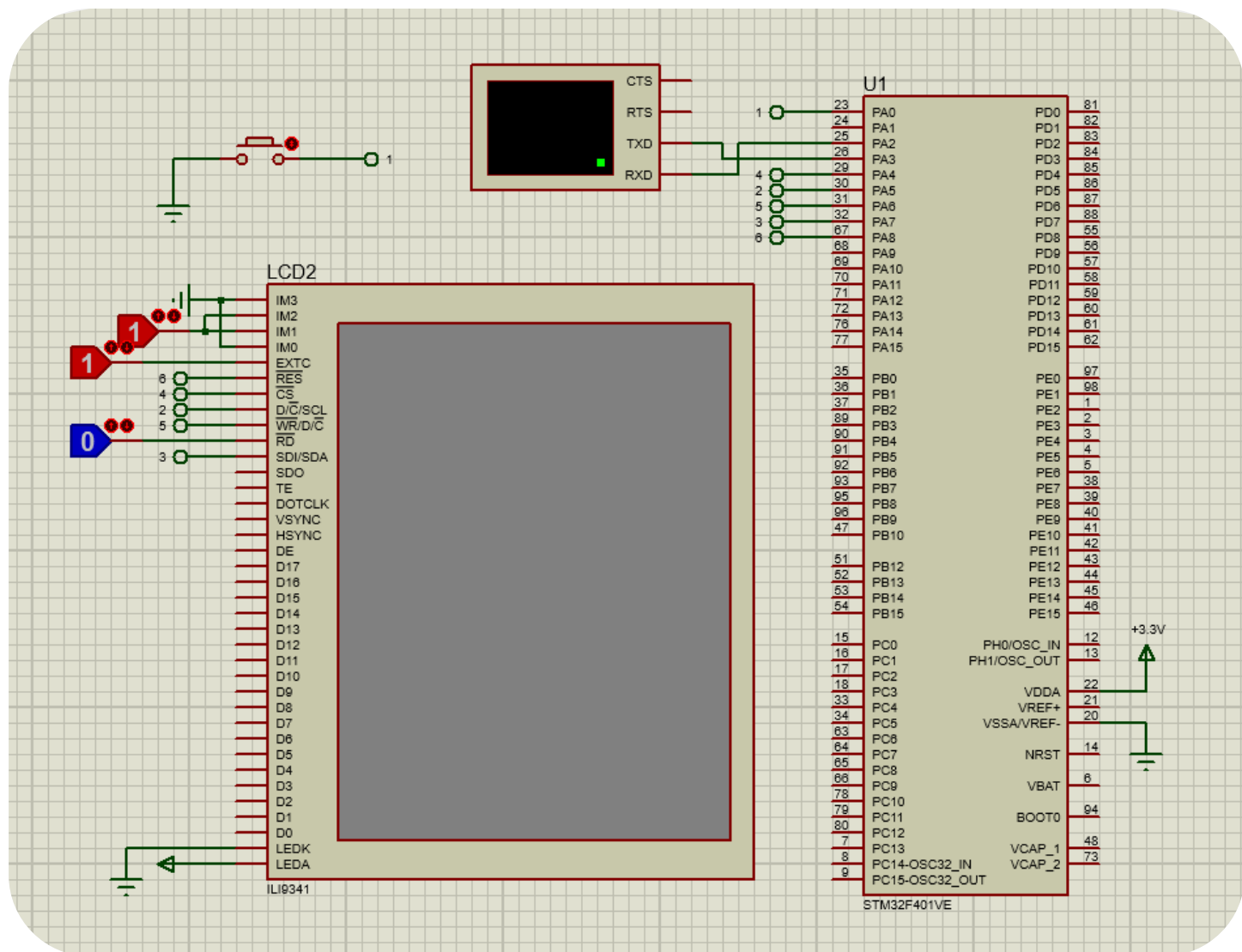
در این فاز، تمامی ماژول‌های توسعه‌یافته در فازهای قبل (هوش مصنوعی، منطق کنترل و درایورهای نمایشگر) بر روی سخت‌افزار مجازی میکروکنترلر STM32F401VE در محیط پروتئوس

(Proteus) ادغام و تست شدند. هدف از این فاز، ارزیابی صحت عملکرد کل سیستم در یک محیط شبیه‌سازی شده قبل از پیاده‌سازی روی برد واقعی است.

مرحله ۱: طراحی شماتیک و اتصالات سخت‌افزاری (Schematic Design)

در این مرحله، تمامی قطعات اصلی در نرم‌افزار پروتئوس چیدمان و سیم‌کشی شدند مطابق:

- واحد پردازش مرکزی (MCU): میکروکنترلر STM32F401VE هسته ARM Cortex-M4 به عنوان مغز متفکر سیستم.
- ارتباط دیباگ (UART): پین‌های PA2 (TX) و PA3 (RX) به ابزار Virtual Terminal متصل شدند تا وضعیت اجرای شبکه عصبی به صورت لحظه‌ای مانیتور شود.
- رابط نمایشگر (LCD Interface): نمایشگر رنگی با درایور ILI9341 به پورت‌های GPIO متصل شد. پین‌های پیکربندی رابط (IM0-IM3) برای تنظیم حالت ارتباطی تنظیم و پین‌های کنترلی مانند CS و RES به پین‌های خروجی میکروکنترلر متصل گردیدند.



مرحله ۲: پیکربندی و راه اندازی سیستم (System Initialization)

پس از اجرای شبیه سازی، سیستم فرآیند راه اندازی را آغاز می کند. همان طور که در تصویر پایین مشاهده می شود، مقادیر اولیه به این شرح است:

- حافظه مدل Act Size: (حافظه لایه‌های فعال‌سازی) برابر با ۴۱,۱۸۸ بایت و Weights Size (حافظه وزن‌های شبکه) برابر ۳,۰۱۲ بایت.
- خروجی ترمینال: پیام AI Engine Initialized Successfully! نشان‌دهنده لود موفق مدل هوش مصنوعی در حافظه میکروکنترلر است.

```
Virtual Terminal
Act Size: 41188, Weights Size: 3012
AI Engine Initialized Successfully!
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 116 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 115 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 120 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 44% | Inference OK | Time: 117 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 67% | Inference OK | Time: 113 ms
State: 1
AI Engine Run | In Size: 9216 | Prob: 67% | Inference OK | Time: 119 ms
State: 1
AI Engine Run | In Size: 9216 | Prob: 79% | Inference OK | Time: 118 ms
State: 2
AI Engine Run | In Size: 9216 | Prob: 79% | Inference OK | Time: 121 ms
State: 3
AI Engine Run | In Size: 9216 | Prob: 95% | Inference OK | Time: 122 ms
State: 3
AI Engine Run | In Size: 9216 | Prob: 95% | Inference OK | Time: 118 ms
State: 3
```

مرحله ۳: تست عملکرد در سناریوی ایمن (Normal Scenario)

در این تست، شبیه‌ساز سنسور مطابق یک تصویر با نویز پس‌زمینه تولید کرد.

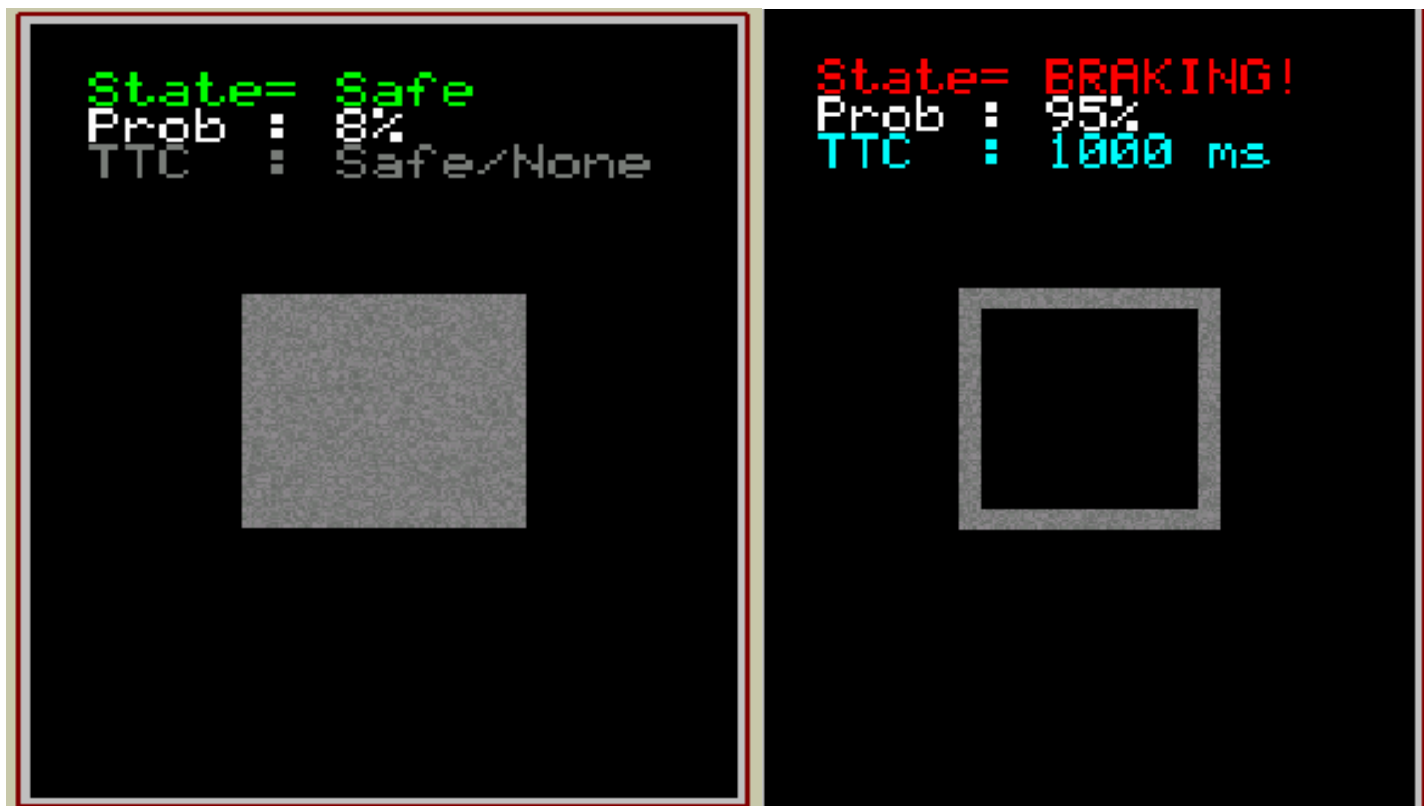
- تحلیل هوش مصنوعی: مدل با احتمال ۸٪ وجود مانع را تشخیص داد که به عنوان Safe تفسیر شد.

- خروجی نمایشگر: متن وضعیت با رنگ سبز و پرچسب Safe نمایش داده شد که نشان‌دهنده شرایط رانندگی عادی است.

مرحله ۴: تست عملکرد در سناریوی بحرانی و ترمز (Braking Scenario)

با فشردن کلید ورودی (شبیه‌سازی نزدیک شدن مانع)، سیستم وارد فاز اضطراری شد:

- تشخیص: هوش مصنوعی احتمال وجود مانع را به ۹۵٪ افزایش داد.
- محاسبه ریسک: شاخص زمان تا برخورد (TTC) برابر با ۱۰۰۰ میلی‌ثانیه محاسبه شد.
- واکنش FSM: ماشین حالت سیستم فوراً به حالت BRAKING! تغییر وضعیت داد و متن خروجی به رنگ قرمز تغییر کرد تا هشدار ترمز اضطراری به راننده داده شود.



مرحله ۵: ارزیابی کارایی (Performance Analysis)

طبق لاگ‌های ترمینال مجازی، هر چرخه استنتاج (Inference) مدل بین ۱۱۵ تا ۱۲۲ میلی ثانیه زمان می‌برد. این زمان نشان می‌دهد که سیستم در نرخ فریم پایینی عمل می‌کند که برای کاربردهای حیاتی اتومبیل‌رانی نیازمند بهینه‌سازی بیشتر است.

نتیجه‌گیری کلی و ابزارهای مورد استفاده

سیستم تشخیص مانع و ترمز اضطراری مبتنی بر TinyML با استفاده از STM32Cube.AI، CMSIS-NN و Proteus روی STM32F401VE شبیه‌سازی شده است. مدل ONNX روی Arm Cortex-M4 مستقر شده است.

این پروژه با موفقیت یک سیستم "هوش مصنوعی در لبه (Edge AI)" را از ایده تا شبیه‌سازی پیش برد. ابزارهای کلیدی استفاده شده در کل پروژه عبارتند از:

- پایتون و **pytorch** : برای توسعه و آموزش مدل هوش مصنوعی.
- **STM32CubeMX & X-CUBE-AI** برای تبدیل مدل به کد C و یکپارچه‌سازی با سخت‌افزار.
- **Keil MDK** برای برنامه‌نویسی سطح پایین و مدیریت درایورهای سخت‌افزاری.
- **Proteus** برای شبیه‌سازی دقیق اتصالات و تست نهایی منطق سیستم.

TinyOD-STM32